

CHAPTER-3-CLASSICAL SEARCH

Wattanapong Suttapak

2 OUTLINE

- Uninformed Search
 - Breadth and Depth First Search
- Informed Search
 - Greedy
 - A* Search
- Adversarial Search
- Local Search
- Optimization

3 DETERMINISTIC AND STOCHASTIC

Deterministic problem	Stochastic problem
- $s = ut + \frac{1}{2} at^2$ - Fibonacci sequence - Sudoku	- Machine Learning problems (Unstable Parameter Initialization) - Image Classification - NLP - Computer Science problems - Weather forecast - Stock forecast

แล้วปัญหาเหล่านี้ละ การเดินทางจาก ICT ไปหน้ามอ วิ่งแข่งชั้น 100 เมตร แข่งแบดมินตัน

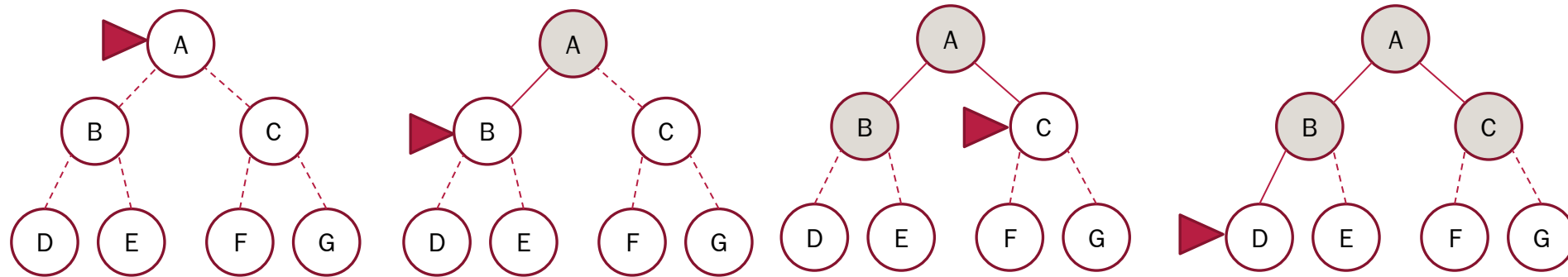
4

UNINFORMED SEARCH

5 SEARCH STRATEGIES

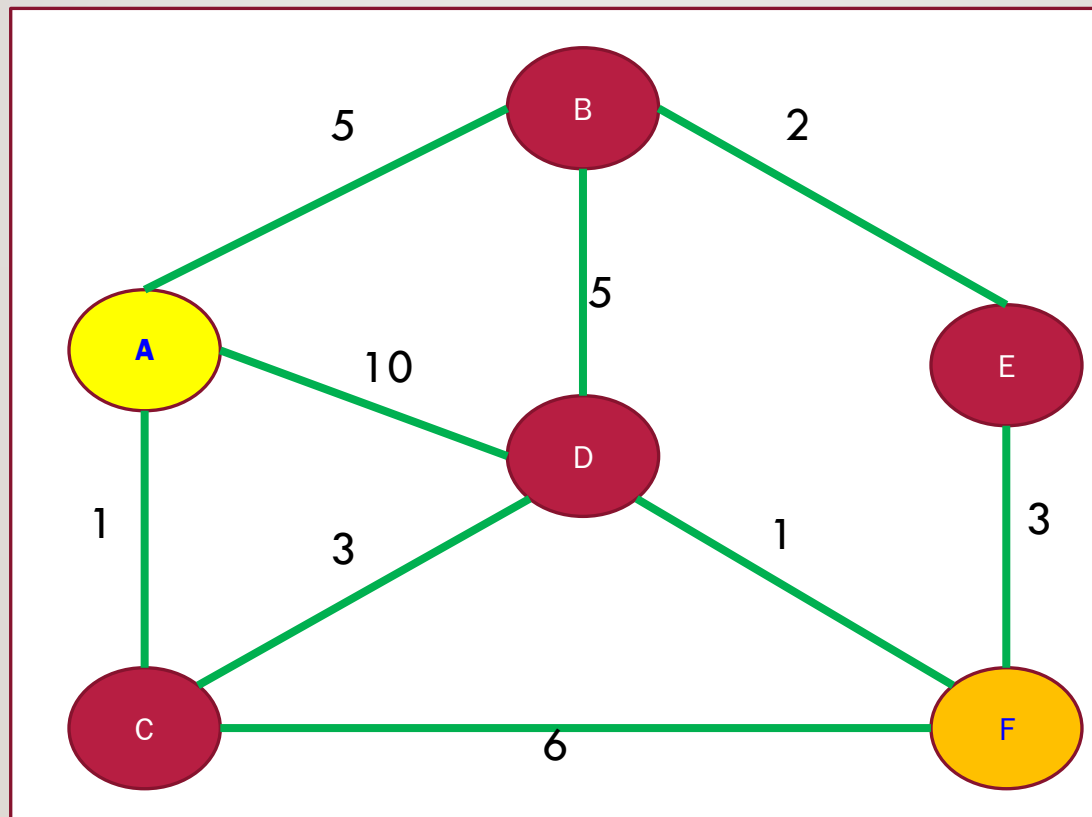
- Uninformed search (blind search)
 - No additional information about states (Weight between edge)
 - Examples **breadth-first search**, uniform-cost search, **depth-first search**, depth-limited search, iterative deepening depth-first search

6 BREADTH-FIRST SEARCH (BFS)



7 BREADTH-FIRST SEARCH

Shortest Path

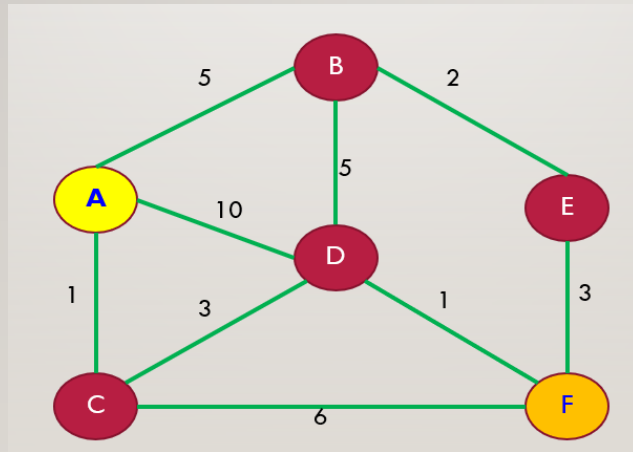


For example

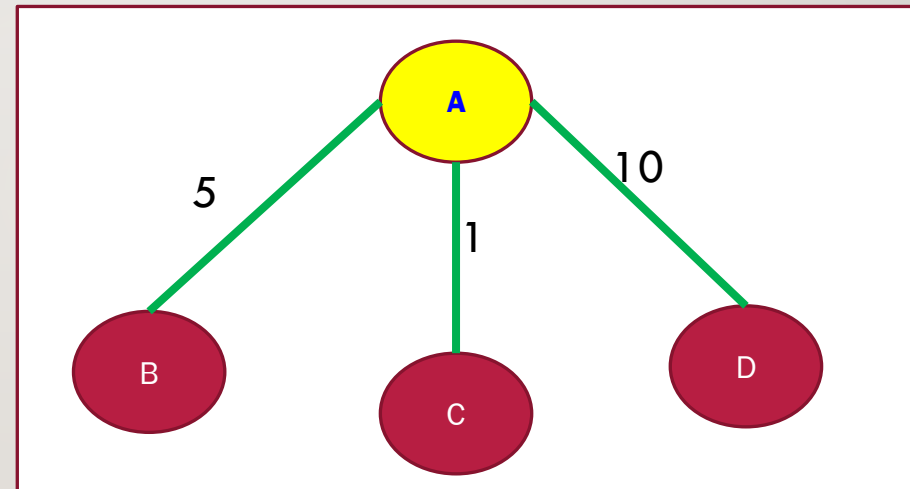
- logistic problem
- express delivery

8 BREADTH-FIRST SEARCH

Shortest Path

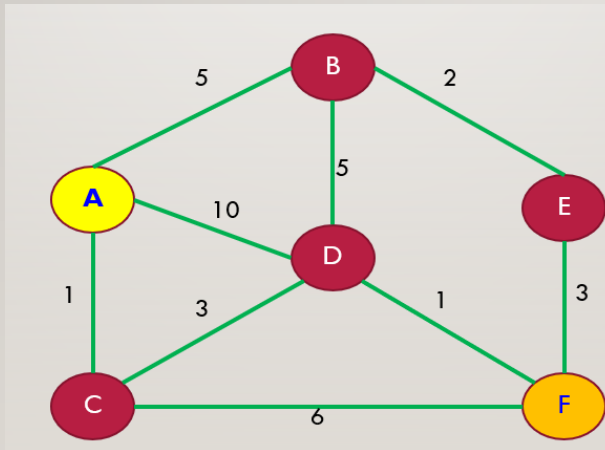


Procedural Searching:

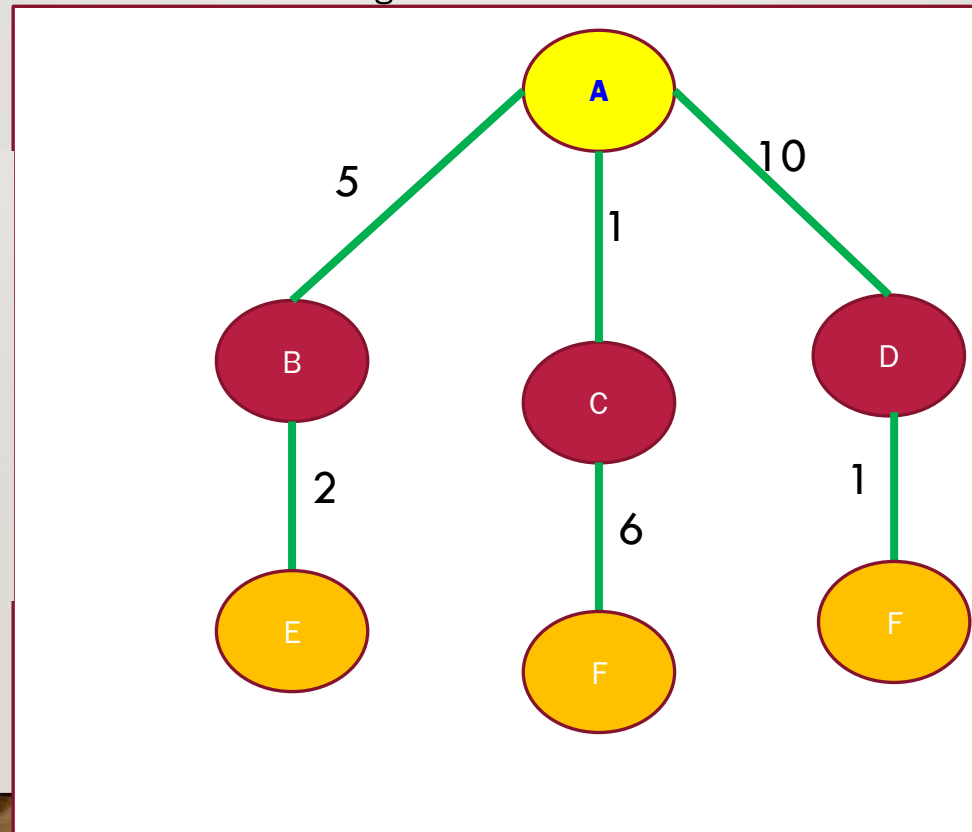


9 BREADTH-FIRST SEARCH

Shortest Path



Procedural Searching:



- Level 1

{[A,B],[A,C],[A,D]}

- Level 2

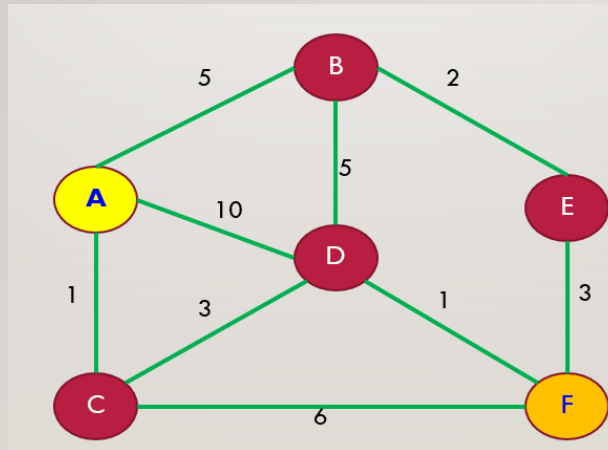
{[B,E]}

- Level 3

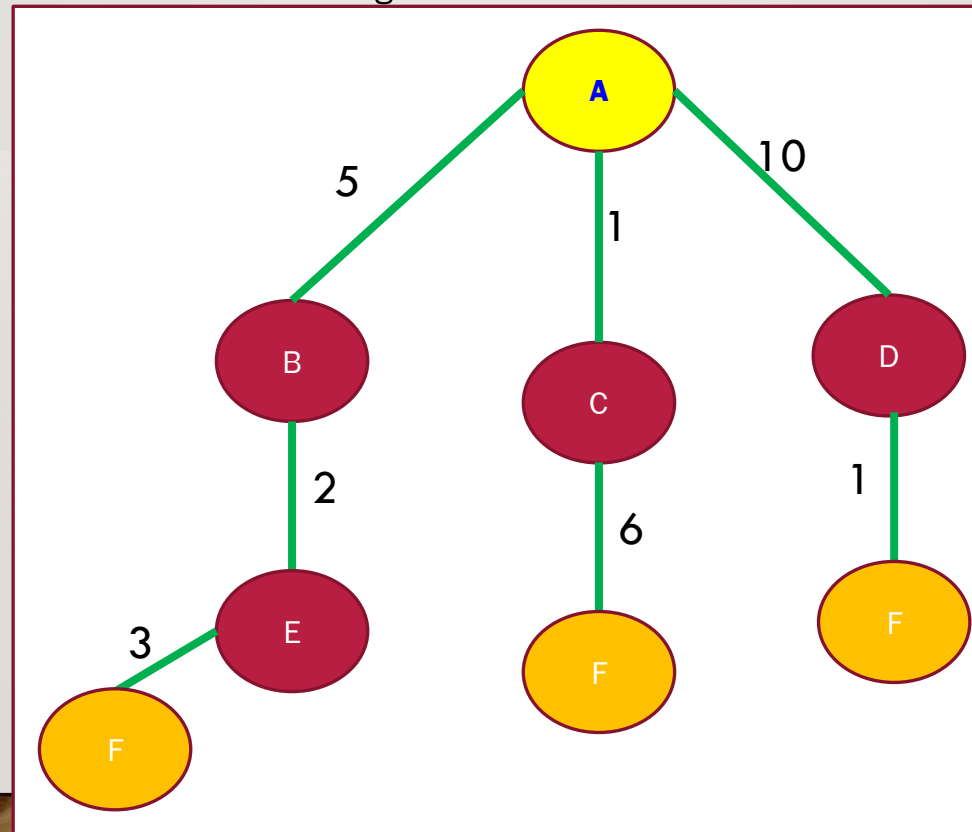
{[E,F]} $\rightarrow w = 5+2+3=10$

10 BREADTH-FIRST SEARCH

Shortest Path



Procedural Searching:



- Level 1

$\{[A,B],[A,C],[A,D]\}$

- Level 2

$\{[B,E]\}$

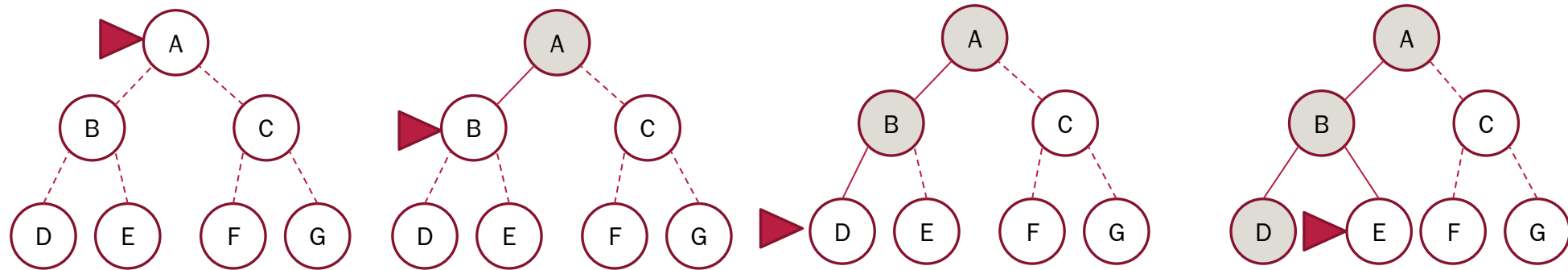
$\{[C,F]\} \rightarrow w = 1+6=7$

$\{[D,F]\} \rightarrow w = 10+1=11$

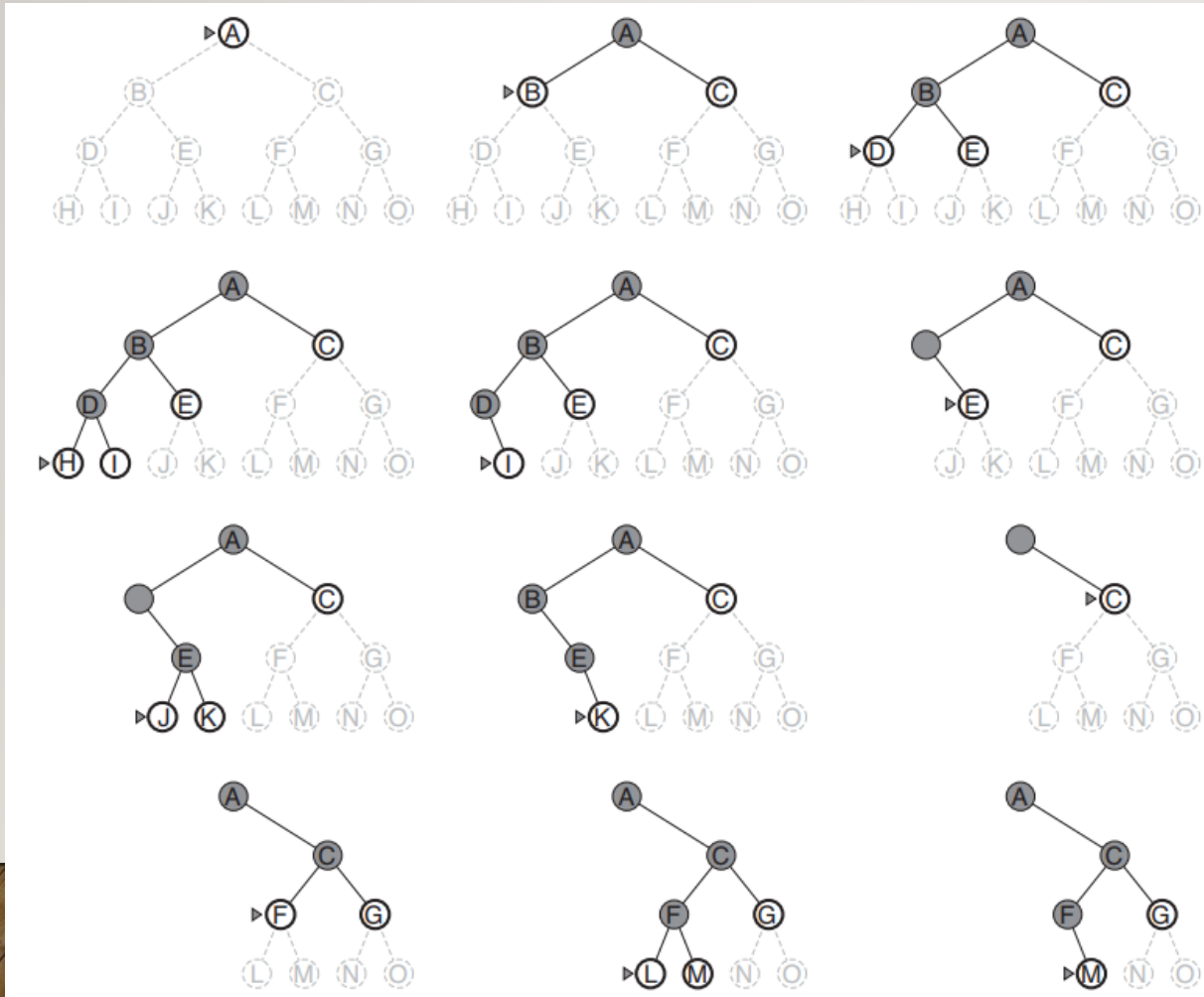
- Level 3

$\{[E,F]\} \rightarrow w = 5+2+3=10$

11 DEPTH-FIRST SEARCH (DFS)



12 DEPTH-FIRST SEARCH



- known as backtracking search

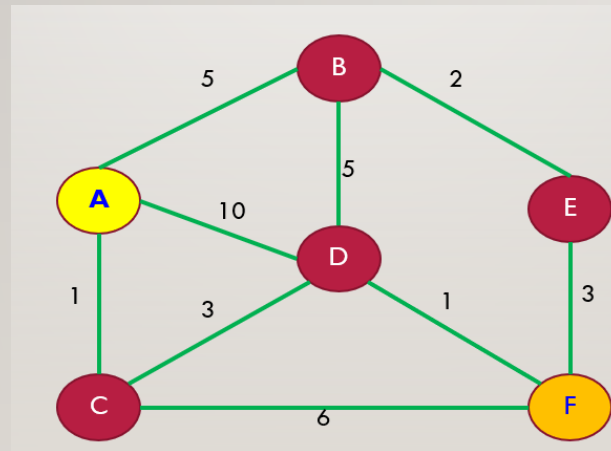
- depth-limited search (limit depth level)

- iterative deepening depth-first search

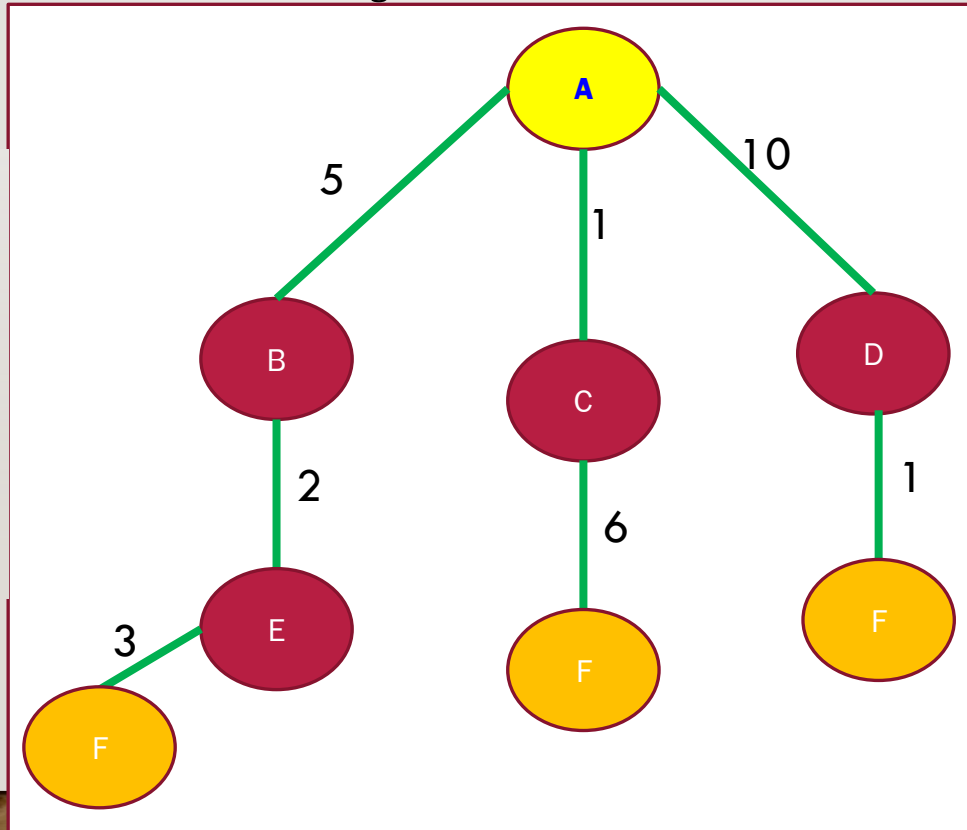
(combine depth and breadth by iteration depth level $\{0, 1, \dots, n\}$)

13 DEPTH-FIRST SEARCH

Shortest Path



Procedural Searching:



{A,B}

{A,B,E}

$$\{A,B,E,F\} = 5 + 2 + 3 = 10$$

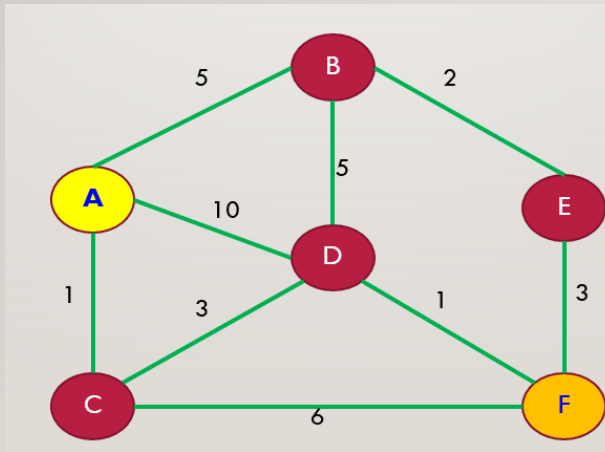
{A,C}

$$\{A,C,F\} = 1 + 6 = 7$$

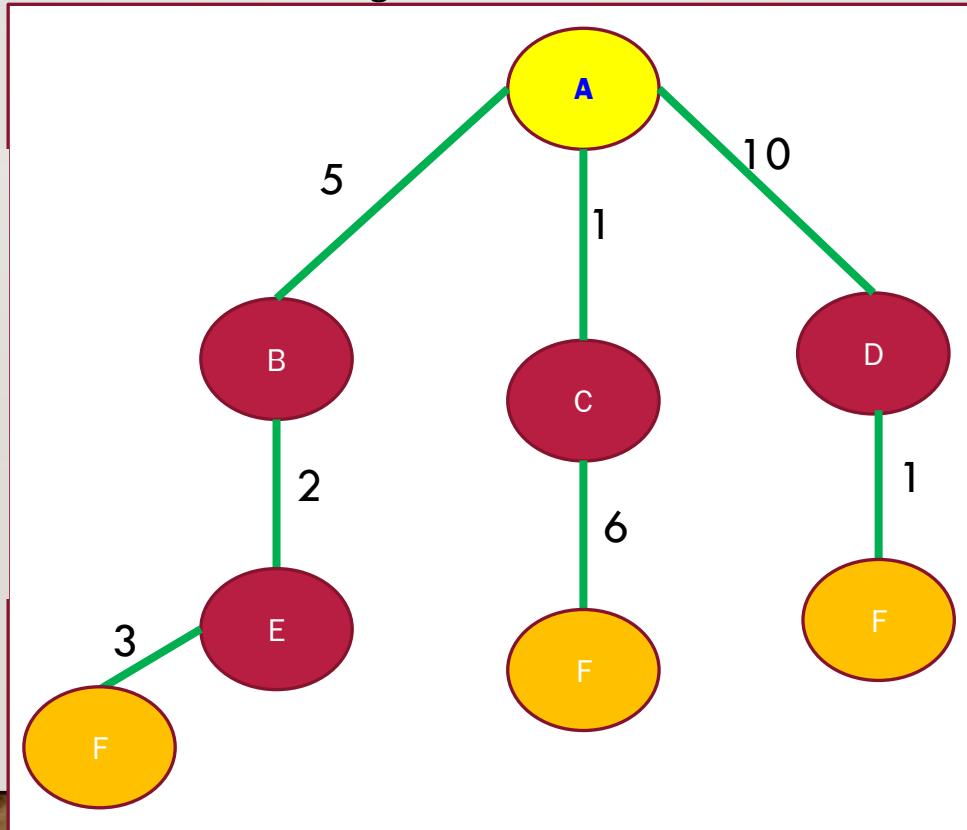
$$\{A,D\} \Rightarrow 10$$

14 DEPTH-LIMITED SEARCH

Shortest Path



Procedural Searching:



Limit depth=2

{A,B}

{A,B,E}

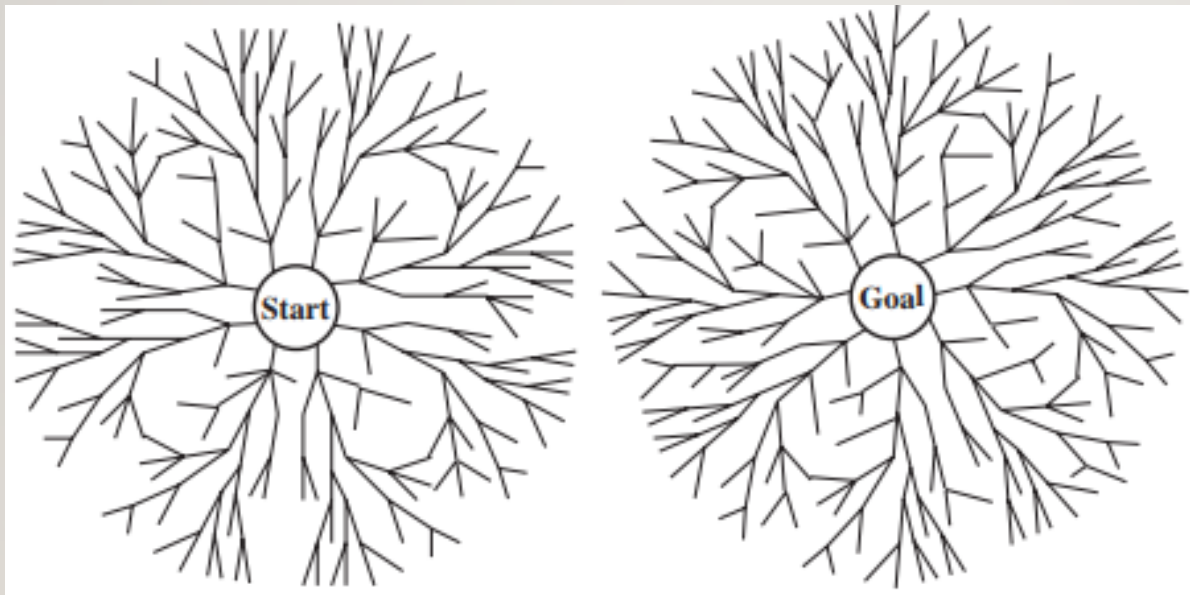
{A,C}

{A,C,F} = 1 + 6 = 7

{A,D} => 10

15 BIDIRECTIONAL SEARCH

- Search from two ways, one from starting point, a latter from goal point



16

INFORMED SEARCH

17 INFORMED SEARCH

- Informed Search หรือเรียกอีกอย่างว่า Heuristic search เป็นวิธีการค้นหาที่มีข้อมูลประกอบการค้นหา
- เป็นวิธีการค้นหาคำตอบ โดยเลือกคำตอบที่ให้ประสิทธิภาพที่ดีที่สุด เช่น คำตอบดีที่สุด หรือ เวลาค้นหาน้อยที่สุด หรือใช้ทรัพยากรน้อยที่สุด จากข้อมูลที่มีให้
- มีการสร้าง Heuristic function เพื่อช่วยในการค้นหา โดยฟังก์ชันนี้จะต้องช่วยตัดสินใจเลือกเส้นทางที่ดีที่สุด โดยใช้ระยะเวลาในการค้นหาน้อยลง เมื่อเทียบกับวิธีแบบ brute force
- ตัวอย่าง
 - **Greedy-best search** (based on heuristic estimate) which is similar to **uniform-cost search** (based on exact path cost) that **prioritized node by optimal cost**
 - **A* search**: Minimizing the total estimated solution cost

18 GREEDY ALGORITHM

- เป็นการค้นหาคำตอบ โดยเลือกคำตอบที่ดีที่สุด ในแต่ละขั้นตอน โดยไม่คำนึงถึงเหตุการณ์ทั้งหมด แต่จะ**ใช้เพียงข้อมูลเท่าที่มี** ดังนั้นคำตอบที่ได้ จะ**ไม่ใช่คำตอบที่ดีที่สุดแบบสากล** (Global optimum solution) หากแต่เป็นเพียง**คำตอบที่ดีที่สุดในระดับท้องถิ่น** (Local optimum solution) ยกเว้นปัญหาที่สามารถเปรียบเทียบทุกความน่าจะเป็นได้ เช่น
 - Kruskal's and Prim's Algorithms: ใช้หาคำตอบของ Minimum Spanning Tree (MST) ในกราฟ
 - Huffman Coding: สำหรับการบีบอัดข้อมูลแบบ lossless ผลลัพธ์ที่ได้จะเป็นคำตอบที่ดีที่สุดแบบ global เสมอ
- ตัวอย่างปัญหาที่ไม่สามารถหาคำตอบได้แบบ **Global optimum**
 - Traveling Salesman Problem (TSP) ปัญหาการหาเส้นทางที่สั้นที่สุดเพื่อให้เดินทางครบทุกจุด
 - Knapsack Problem ปัญหาการเลือกสิ่งของใส่ลงในถุงให้ได้มูลค่าที่สูงที่สุด

19 PROBLEM EXAMPLES

- ทอนเงิน Coin change
- ไดสตรา Dijkstra algorithm
- การเดินทางของเซลแมน TSP

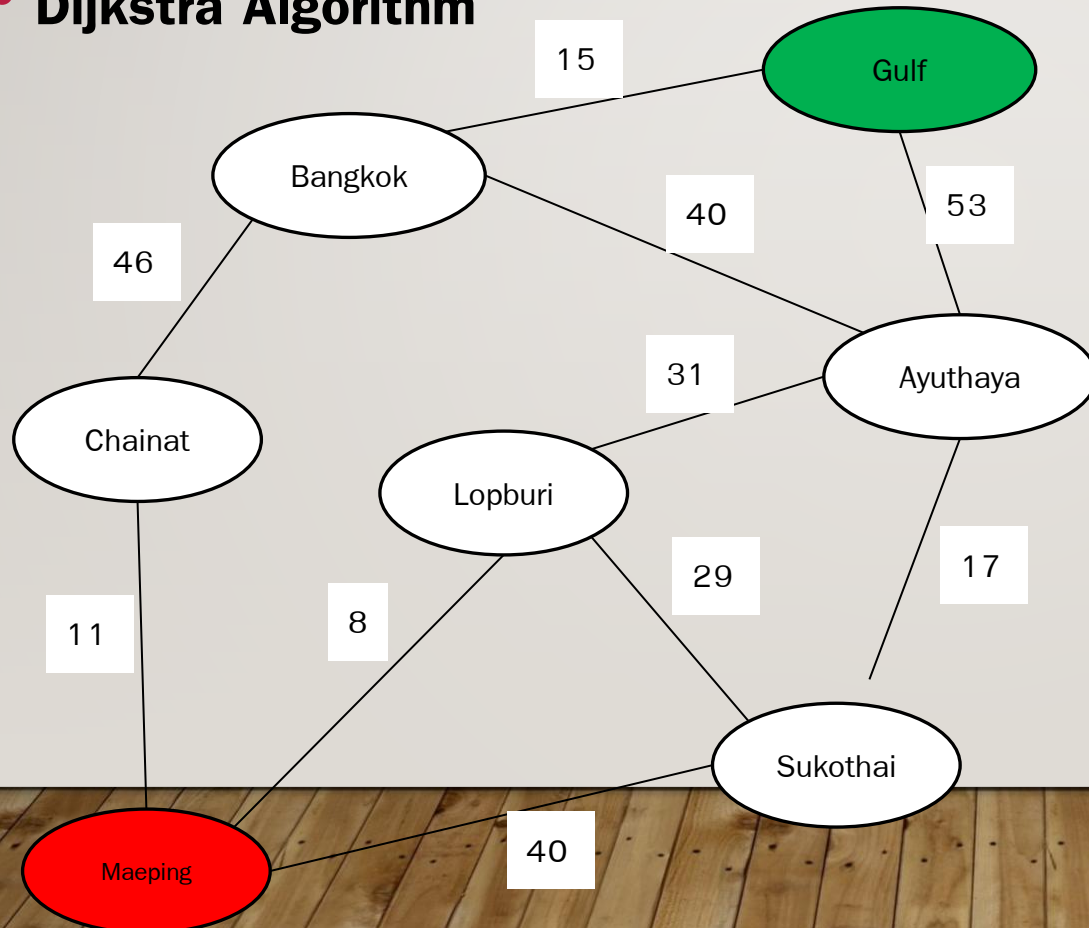
20 GREEDY ALGORITHM

- **Coin Change (ตัวอย่างปัญหาการทอนเงิน)**

- สมมติมีเหรียญ 10, 5, 2, 1 บาท และธนบัตร 1000, 500, 100, 50, 20 บาท
- จะมีวิธีทอนเงินให้กับลูกค้าหลังจากซื้อของไป 37 บาท โดยให้ธนบัตร 100 บาท ไปอย่างไร
- วิธีการ
 - ต้องทอนเงิน $100 - 37 = 63$
 - เปลี่ยนเงินจาก 63 บาท ไปเป็นธนบัตร และเหรียญ เรียงลำดับจากมากไปน้อย
 - ธนบัตร 1x50
 - เหรียญ 1x10, 1x2, 1x1

21 GREEDY ALGORITHM

- **Dijkstra Algorithm**

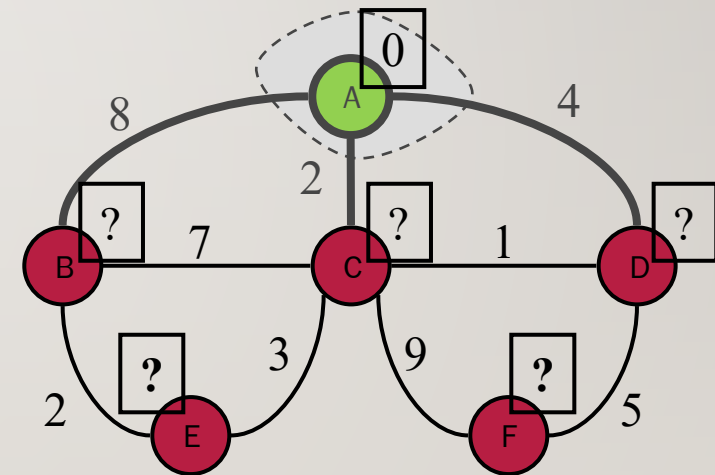


Edger Dijkstra
Turing award 1972

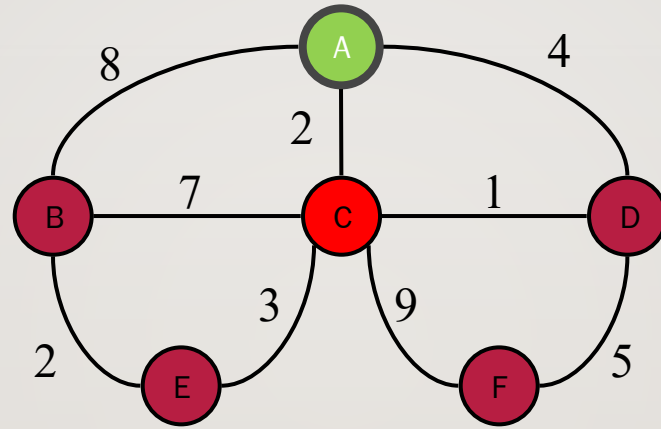
22

DIJKSTRA'S ALGORITHM

- วิธีหาระยะทางที่สั้นที่สุด(shortest path)
- สร้างบนพื้นฐานของ greedy algorithm
- ตัวอย่างการหาระยะทางที่สั้นที่สุดจากจุด A ไปยังทุก ๆ จุด



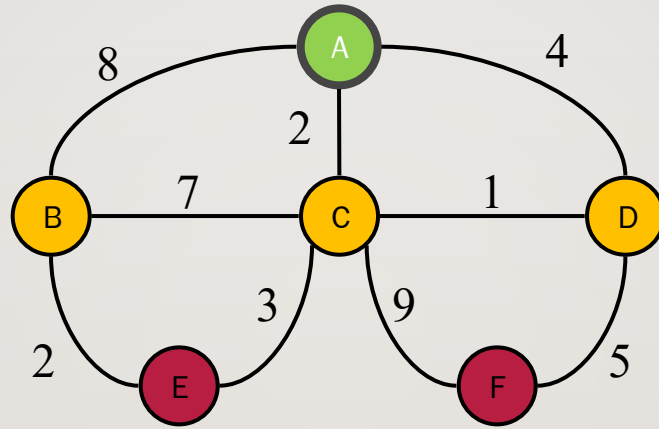
23



Vertex	A	B	C	D	E	F
Distance	0	∞	∞	∞	∞	∞

24

- คำนวณระยะทางจากจุด A ไปทุกจุดที่มีการเชื่อมต่อ (Adjacent node)

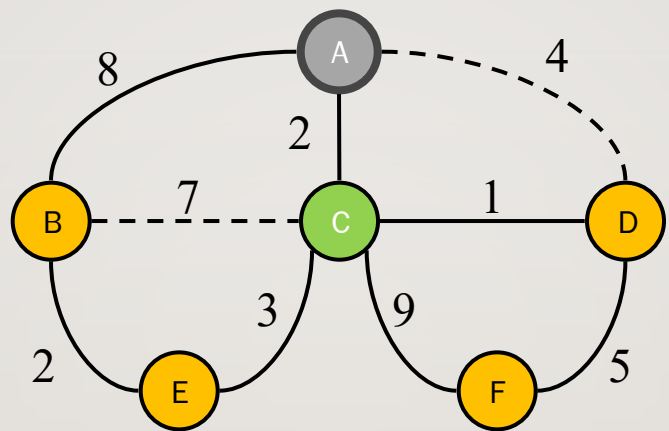


Path	Distance
A -> C	2
A -> B	8
A -> D	4

Vertex	B	C	D	E	F
Distance	8	2	4		
From Vertex	A	A	A		

25

- เลือกจุดหลักต่อไปในการค้นหา
- โดยเลือกจุดที่มีระยะทางสั้นที่สุดจากจุด A
- ดังนั้นเลือกจุด C

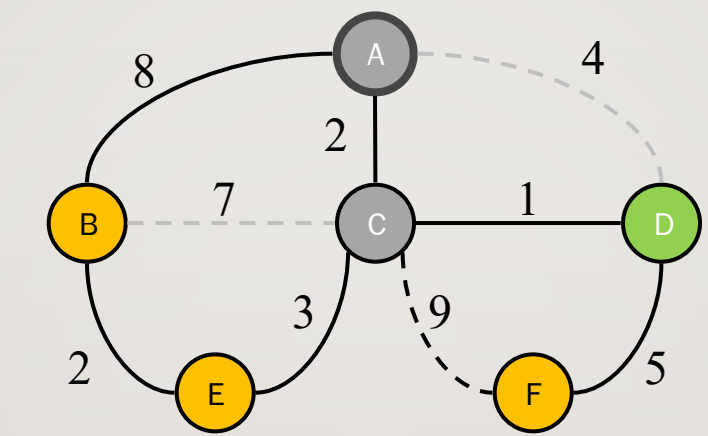


Vertex	B	D	E	F
Distance	9	3	5	11
From Vertex	C	C	C	C

Path	Distance
A -> C	2
A -> B	8
A -> C -> B	9
A -> D	4
A -> C -> D	3
A -> C -> E	5
A -> C -> F	11

26

- เลือกจุดหลักต่อไปในการค้นหา
- โดยเลือกจุดที่มีระยะทางสั้นที่สุดจากจุด C
- ยกเว้นจุดที่เคยกำหนดเป็นจุดหลักมาแล้ว (A)
- ดังนั้นเลือกจุด D เป็นจุดหลัก
- D -> F สั้นกว่า D -> C -> F

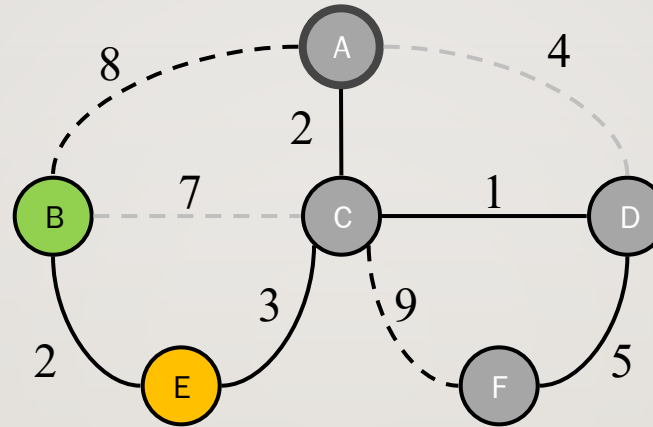


Vertex	F
Distance	8
From Vertex	D

Path	Distance
A -> C	2
A -> B	8
A -> C -> D	3
A -> C -> E	3
A -> C -> F	11
A -> C -> D -> F	8

27

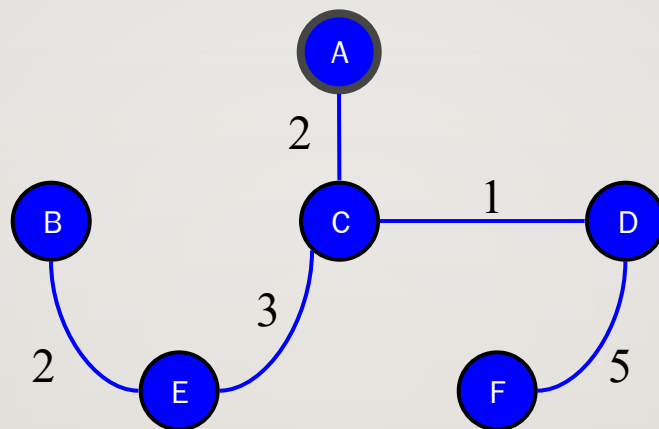
- เลือกจุดหลักต่อไปในการค้นหา ที่ไม่ใช่ {A,C,D}
- เลือกจุด F แต่จุด F ไม่มีจุดเชื่อมต่ออื่นที่เหลือแล้ว
- เลื่อนจุดต่อไปเป็นจุด B
- B -> A ยาวกว่า B -> E -> C -> A



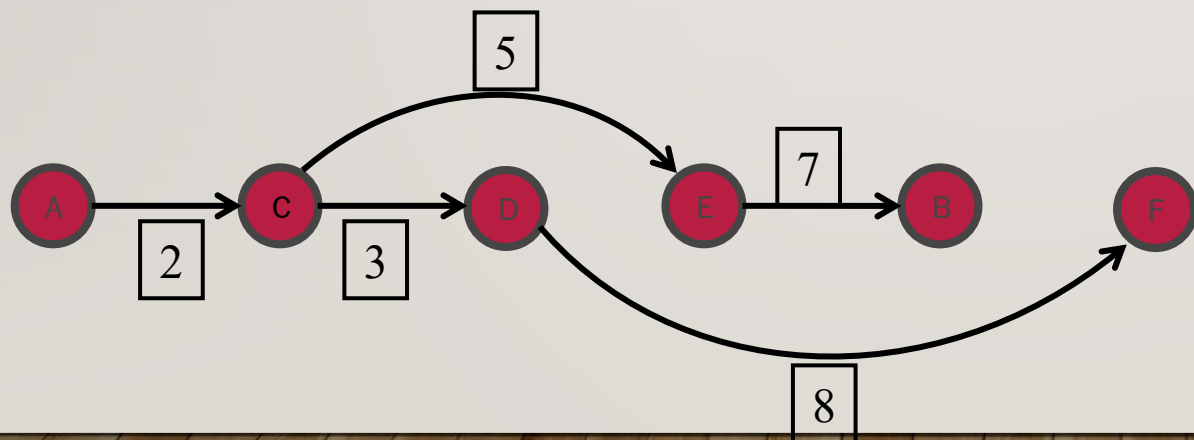
Vertex	B	E
Distance	7	5
From Vertex	E	C

Path	Distance
A -> C	2
A -> B	8
A -> C -> E - B	7
A -> C -> D	3
A -> C -> E	3
A -> C -> D -> F	8

เส้นทาง A - C - E - B สั้นกว่าเส้นทาง A - B
 และไม่มีเส้นทางอื่นจากจุด E ที่มีระยะทางสั้นกว่าเส้นทาง A - C - E



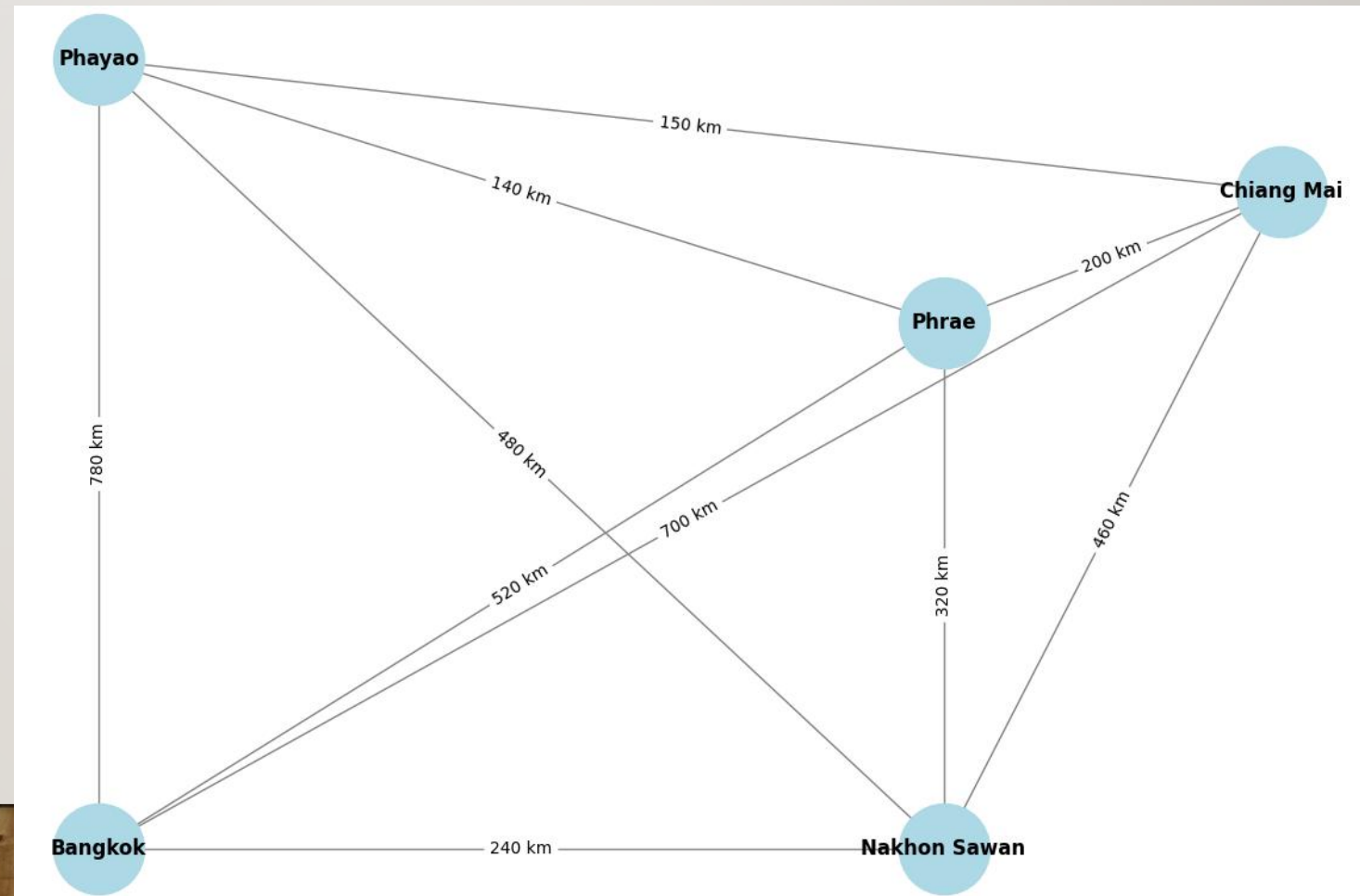
Path	Distance
A -> C	2
A -> C -> E -> B	7
A -> C -> D	3
A -> C -> E	5
A -> C -> D -> F	8



29 TRAVELLING SALESMAN PROBLEM

หาเส้นทางที่สั้นที่สุดในการเดินทางผ่าน ให้ครบทุกจุด (ไม่สามารถเดินทางกลับเส้นทางเดิมได้)

- Bangkok
- Nakhon Sawan
- Phrae
- Chiang Mai
- Phayao



30 TRAVELLING SALESMAN PROBLEM

- **1. States:** แต่ละสถานะ แสดงถึงระยะทางระหว่างเมือง
- **2. Initial State:** กำหนดจุดเริ่ม เช่น เริ่มต้นจาก Bangkok
- **3. Actions:** เดินทางจากเมืองเริ่มต้น หรือเมืองปัจจุบัน ไปยังเมืองที่ยังไม่เคยผ่าน
- **4. Goal State:** มีการเดินทางผ่านทุก ๆ เมือง และกลับไปยังจุดเริ่มต้น
- **5. Path Cost:** ระยะทางรวมทุก ๆ เมือง และเส้นทางที่เดินทางจากจุดเริ่มต้นไปถึงเมืองสุดท้าย และเดินทางกลับ

31 TRAVELLING SALESMAN PROBLEM

- ตัวอย่าง
- 1. **States:** Dict of {C,V} เช่น {Bangkok,{Bangkok, Nakhon Sawan}}
- 2. **Initial State:** {Bangkok}
- 3. **Actions:** {Bangkok,Nakhon Sawan} {Bangkok,Phrae} {Bangkok,Chiang Mai} {Bangkok,Phayao}
- 4. **Goal State:** {Bangkok,{Bangkok,Nakhon Sawan,Phrae,Chiang Mai,Phayao}}
- 5. **Path Cost:** {Bangkok,Nakhon Sawan} + ...

32 TRAVELLING SALESMAN PROBLEM

- แก้ปัญหาด้วย Greedy Algorithm (Nearest Neighbor Search)
 1. กำหนดจุดค้นหา เป็นจุดเริ่มต้น
 2. คำนวณระยะทางจากจุดค้นหาไปยังทุกจุดที่ไม่เคยผ่าน
 3. เลือกระยะทางที่สั้นที่สุด
 4. เปลี่ยนจุดค้นหา เป็นจุดที่มีระยะสั้นที่สุด
 5. ทำซ้ำ 2 – 4 จนกว่าจะเหลือจุดหมายสุดท้าย

33 TRAVELLING SALESMAN PROBLEM

```
2  # Define the cities and distances between them
3  cities = ["Bangkok", "Nakhon Sawan", "Phrae", "Chiang Mai", "Phayao"]
4  distances = {
5      ("Bangkok", "Nakhon Sawan"): 240,
6      ("Bangkok", "Phrae"): 520,
7      ("Bangkok", "Chiang Mai"): 700,
8      ("Bangkok", "Phayao"): 780,
9      ("Nakhon Sawan", "Phrae"): 320,
10     ("Nakhon Sawan", "Chiang Mai"): 460,
11     ("Nakhon Sawan", "Phayao"): 480,
12     ("Phrae", "Chiang Mai"): 200,
13     ("Phrae", "Phayao"): 140,
14     ("Chiang Mai", "Phayao"): 150
15 }

50 # Start at Bangkok
51 start_city = "Bangkok"
52 route, total_distance = greedy_tsp(start_city, cities, distances)
53
54 print(route)
55 print(total_distance)
```

34 TRAVELLING SALESMAN PROBLEM

```
16
17 # Create a function to find the nearest neighbor
18 def find_nearest_neighbor(current_city, unvisited_cities, distances):
19     nearest_city = None
20     min_distance = float('inf')
21     for city in unvisited_cities:
22         distance = distances.get((current_city, city)) \
23             or distances.get((city, current_city))
24         if distance < min_distance:
25             min_distance = distance
26             nearest_city = city
27     return nearest_city, min_distance
```

```
28
29 # Greedy algorithm to solve TSP using the Nearest Neighbor approach
30 def greedy_tsp(start_city, cities, distances):
31     unvisited_cities = set(cities)
32     unvisited_cities.remove(start_city)
33     current_city = start_city
34     route = [start_city]
35     total_distance = 0
36
37     while unvisited_cities:
38         next_city, distance = \
39             find_nearest_neighbor(current_city, unvisited_cities, distances)
40         route.append(next_city)
41         total_distance += distance
42         current_city = next_city
43         unvisited_cities.remove(next_city)
44
45     # Return to the starting city
46     return_distance = distances.get((current_city, start_city)) \
47         or distances.get((start_city, current_city))
48     route.append(start_city)
49     total_distance += return_distance
50
51     return route, total_distance
```

35 TRAVELLING SALESMAN PROBLEM

แก้ปัญหาด้วย Greedy Algorithm (Best First Search) = GBFS

1. กำหนดจุดค้นหา (A) เป็นจุดเริ่มต้น
2. คำนวณระยะทางจากจุดค้นหา (A) ไปยังทุกจุดที่ไม่เคยผ่าน (B)
3. สร้างฟังก์ชัน Heuristic คำนวณระยะทางระหว่างจุด เริ่มต้น ไปยังจุดอื่น ๆ ใกล้เคียง
4. แต่ละจุดที่เข้าถึง (B) ทำการคำนวณค่า Heuristic (B) ไปยังทุกจุดที่ยังไม่เคยผ่าน (C,D, ...)
5. เลือกระยะทางที่สั้นที่สุด
6. เปลี่ยนจุดค้นหา เป็นจุดต่อไปที่มีระยะสั้นที่สุด
7. ทำซ้ำ 4 – 6 จนกว่าจะเหลือจุดหมายสุดท้าย

36 TRAVELLING SALESMAN PROBLEM

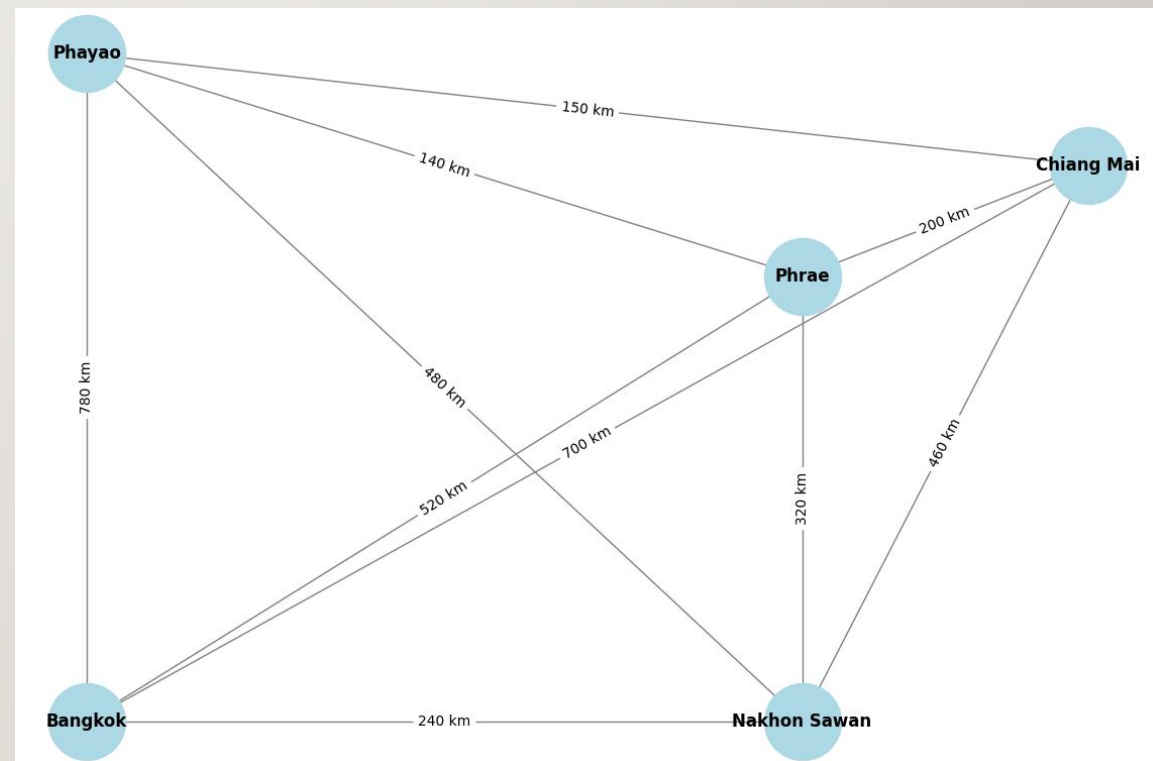
```
68 def heuristic(current_city, unvisited_cities, distances):
69     if not unvisited_cities:
70         return 0
71     nearest_distance = float('inf')
72     for city in unvisited_cities:
73         distance = distances[frozenset([current_city, city])]
74         if distance < nearest_distance:
75             nearest_distance = distance
76     return nearest_distance
```

```
79 def greedy_best_first_search(start_city, cities, distances):
80     unvisited_cities = set(cities)
81     unvisited_cities.remove(start_city)
82     current_city = start_city
83     route = [start_city]
84     total_distance = 0
85
86     while unvisited_cities:
87         # Find the nearest neighbor using the heuristic
88         next_city = None
89         min_heuristic = float('inf')
90         for city in unvisited_cities:
91             distance = distances[frozenset([current_city, city])]
92             h = heuristic(city, unvisited_cities - {city}, distances) + distance
93             if h < min_heuristic:
94                 min_heuristic = h
95                 next_city = city
96
97         # Move to the next city
98         route.append(next_city)
99         total_distance += distances[frozenset([current_city, next_city])]
100        current_city = next_city
101        unvisited_cities.remove(next_city)
102
103        # Return to the starting city
104        return_distance = distances[frozenset([current_city, start_city])]
105        route.append(start_city)
106        total_distance += return_distance
```

37 TRAVELLING SALESMAN PROBLEM

หาเส้นทางที่สั้นที่สุดในการเดินทางผ่าน ให้ครบทุกจุด (ไม่สามารถเดินทางกลับเส้นทางเดิมได้)

คำตอบเหมือนกัน ทั้ง NNS และ DBFS
['Bangkok', 'Nakhon Sawan', 'Phrae', 'Phayao',
'Chiang Mai', 'Bangkok']
1550



38 TRAVELLING SALESMAN PROBLEM

หาเส้นทางที่สั้นที่สุดในการเดินทางผ่าน ให้ครบทุกจุด (ไม่สามารถเดินทางกลับเส้นทางเดิมได้)

NNS

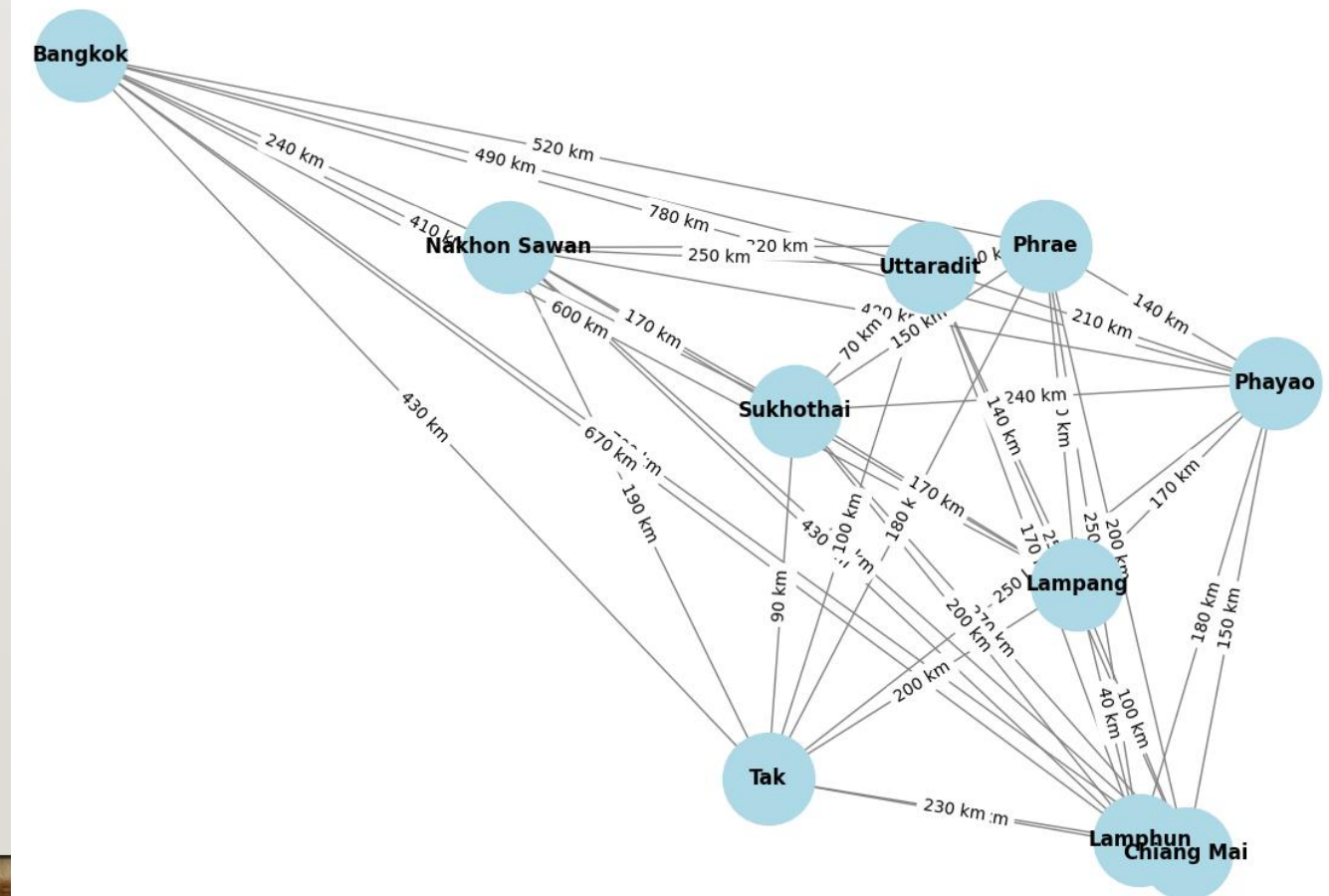
['Bangkok', 'Nakhon Sawan', 'Sukhothai', 'Uttaradit',
'Tak', 'Phrae', 'Phayao', 'Chiang Mai', 'Lamphun',
'Lampang', 'Bangkok']

1720

DBFS

['Bangkok', 'Nakhon Sawan', 'Sukhothai', 'Uttaradit',
'Lampang', 'Lamphun', 'Chiang Mai', 'Phayao', 'Phrae',
'Tak', 'Bangkok']

1590



39

("Bangkok", "Nakhon Sawan"): 240,
("Bangkok", "Phrae"): 520,
("Bangkok", "Chiang Mai"): 700,
("Bangkok", "Phayao"): 780,
("Bangkok", "Lampang"): 600,
("Bangkok", "Lamphun"): 670,
("Bangkok", "Uttaradit"): 490,
("Bangkok", "Sukhothai"): 410,
("Bangkok", "Tak"): 430,

("Nakhon Sawan", "Phrae"): 320,
("Nakhon Sawan", "Chiang Mai"): 460,
("Nakhon Sawan", "Phayao"): 480,
("Nakhon Sawan", "Lampang"): 360,
("Nakhon Sawan", "Lamphun"): 430,
("Nakhon Sawan", "Uttaradit"): 250,
("Nakhon Sawan", "Sukhothai"): 170,
("Nakhon Sawan", "Tak"): 190,
("Phrae", "Chiang Mai"): 200,
("Phrae", "Phayao"): 140,
("Phrae", "Lampang"): 220,
("Phrae", "Lamphun"): 250,
("Phrae", "Uttaradit"): 110,
("Phrae", "Sukhothai"): 150,
("Phrae", "Tak"): 180,

("Chiang Mai", "Phayao"): 150,
("Chiang Mai", "Lampang"): 100,
("Chiang Mai", "Lamphun"): 30,
("Chiang Mai", "Uttaradit"): 250,
("Chiang Mai", "Sukhothai"): 270,
("Chiang Mai", "Tak"): 280,
("Phayao", "Lampang"): 170,
("Phayao", "Lamphun"): 180,
("Phayao", "Uttaradit"): 210,
("Phayao", "Sukhothai"): 240,
("Phayao", "Tak"): 250,
("Lampang", "Lamphun"): 40,
("Lampang", "Uttaradit"): 140,
("Lampang", "Sukhothai"): 170,
("Lampang", "Tak"): 200,

("Lamphun", "Uttaradit"): 170,
("Lamphun", "Sukhothai"): 200,
("Lamphun", "Tak"): 230,
("Uttaradit", "Sukhothai"): 70,
("Uttaradit", "Tak"): 100,
("Sukhothai", "Tak"): 90,

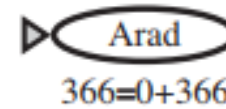


40 A* SEARCH

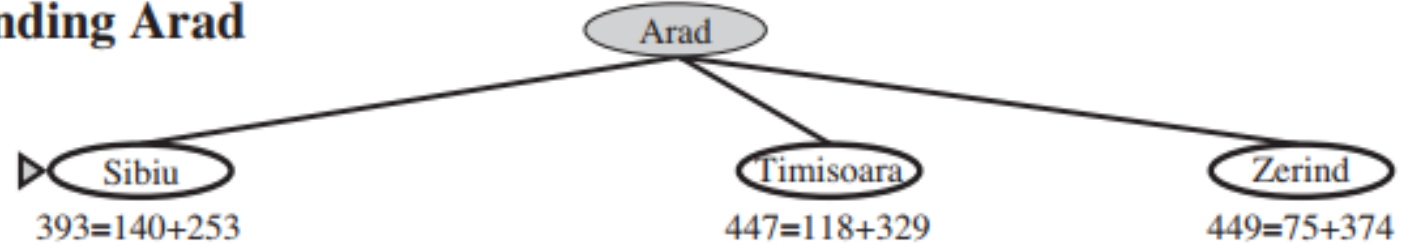
- นำข้อดีของทั้ง **uniform-cost search** (Depth or breadth first search) และ **greedy best-first search** มาใช้งาน
- สร้าง Open List เพื่อเก็บ node เริ่มต้น และ node ทั้งหมดที่ได้จากการสำรวจ (node traversal)
- สร้าง Close List เพื่อเก็บ node ที่ได้ผ่านการสำรวจแล้ว กำหนดให้ค่าเริ่มต้นเป็นเซตว่าง []
- แต่ละ node ของ Open List ทำการคำนวณสองค่าดังนี้
 - **$g(n)$** : คำนวณ cost ของเส้นทาง (steps) จาก node เริ่มต้น ถึง node ปัจจุบัน
 - **$h(n)$** : คำนวณ cost จาก node ปัจจุบัน เทียบกับ node เป้าหมาย (**Heuristic function = calculate smallest path**)
- ทำการเลือก node ที่มีค่า $f(n)$ น้อยที่สุด และทำการลบ node ทั้งหมดที่ทำเปรียบเทียบจาก Open List และย้ายไปยัง Close List
- ทำซ้ำจนกว่าจะถึงเป้าหมาย

41 A* SEARCH

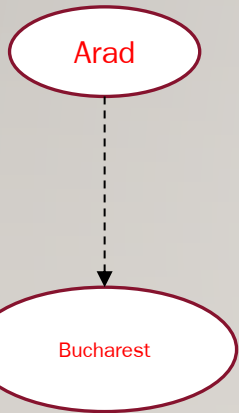
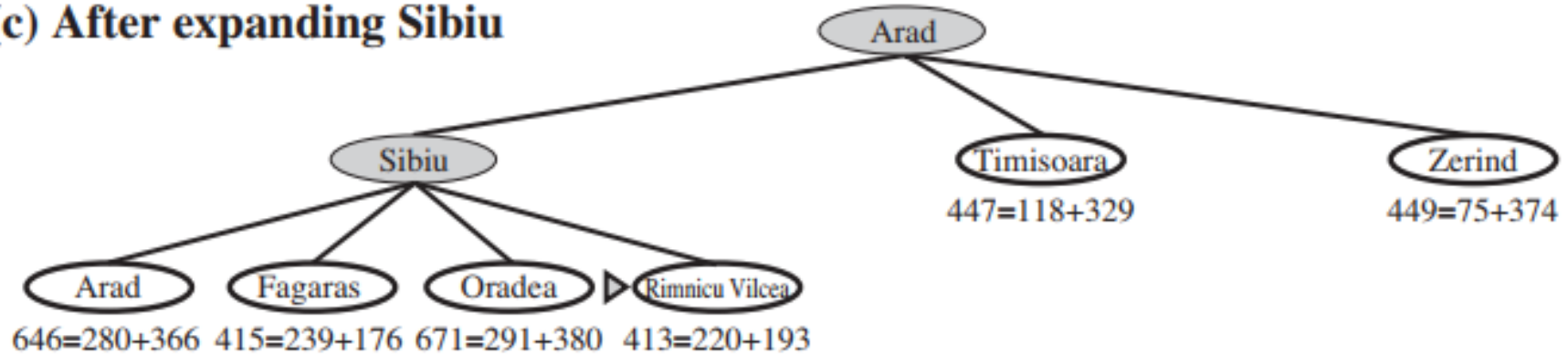
(a) The initial state



(b) After expanding Arad

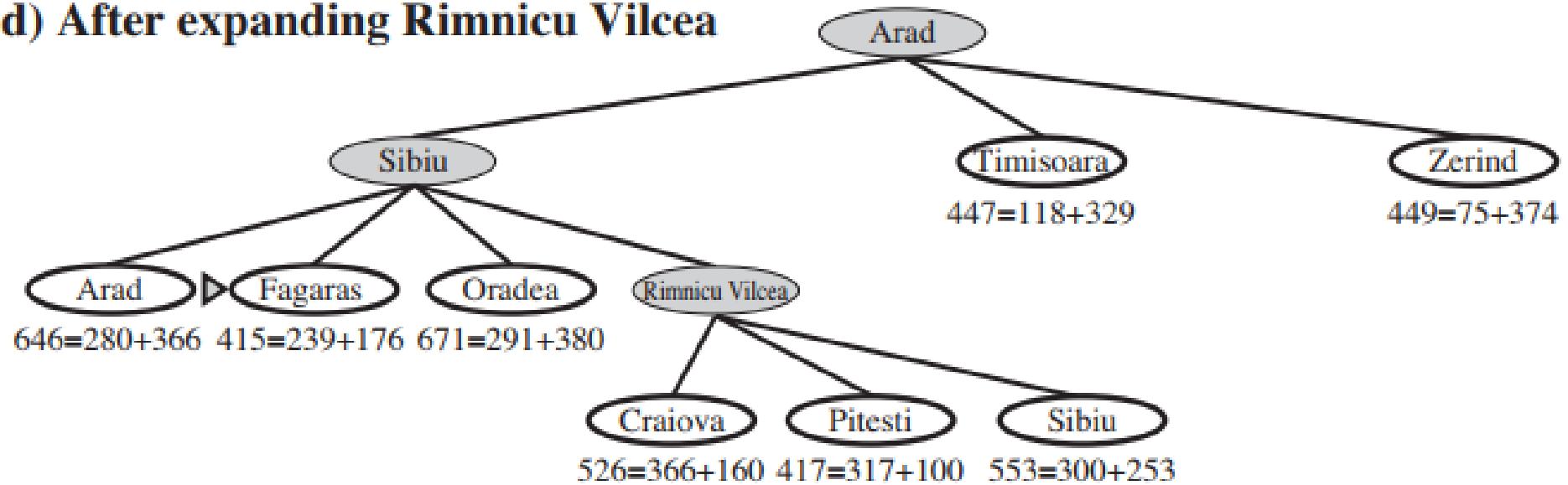


(c) After expanding Sibiu



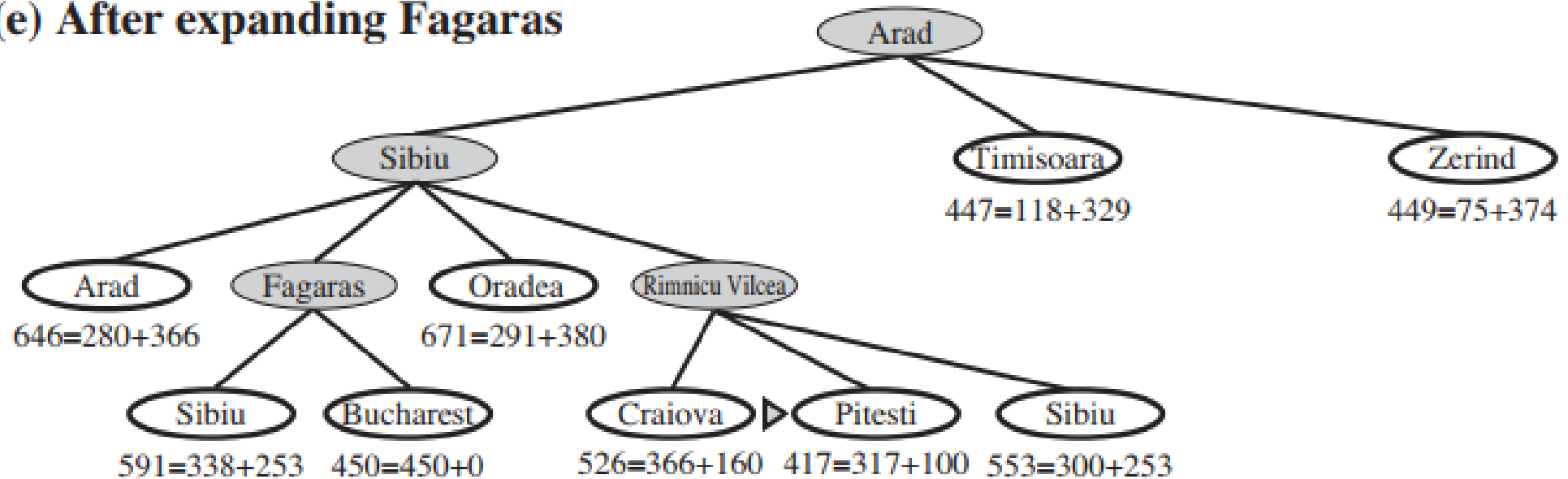
42 A* SEARCH

(d) After expanding Rimnicu Vilcea



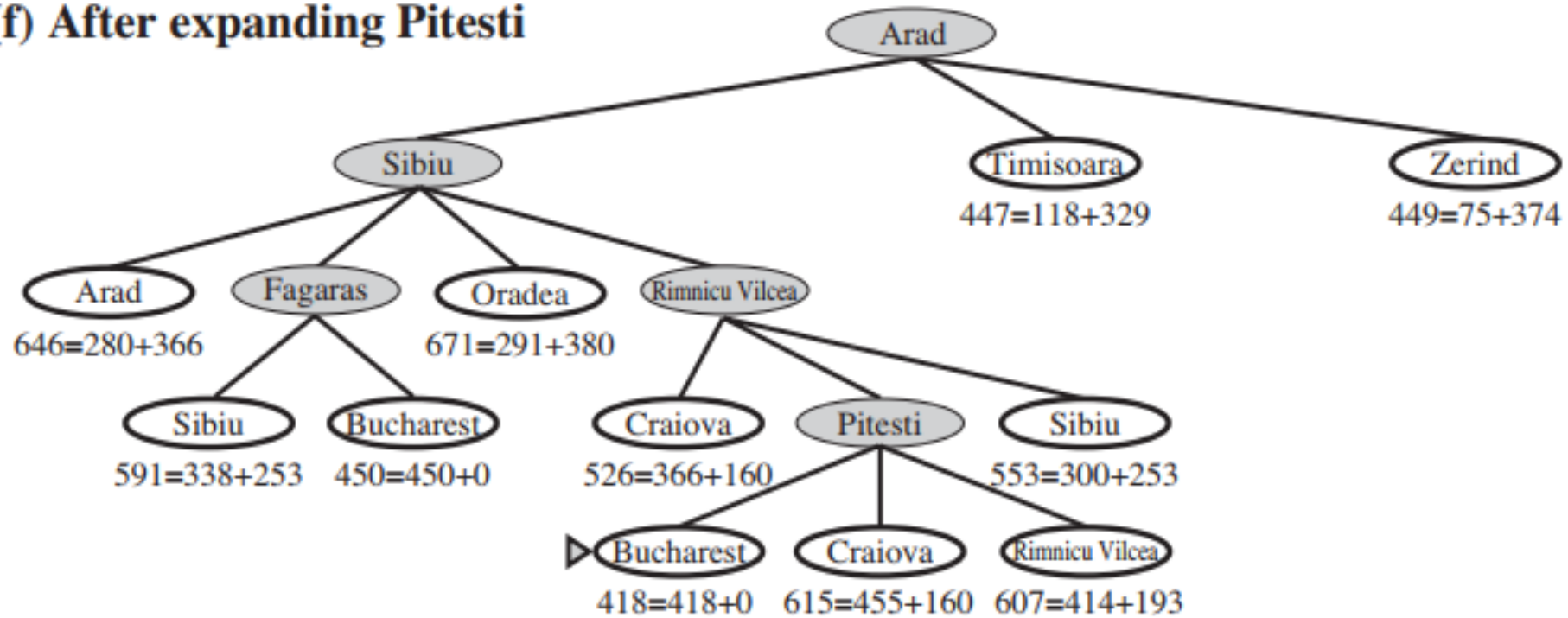
43 A* SEARCH

(e) After expanding Fagaras



44 A* SEARCH

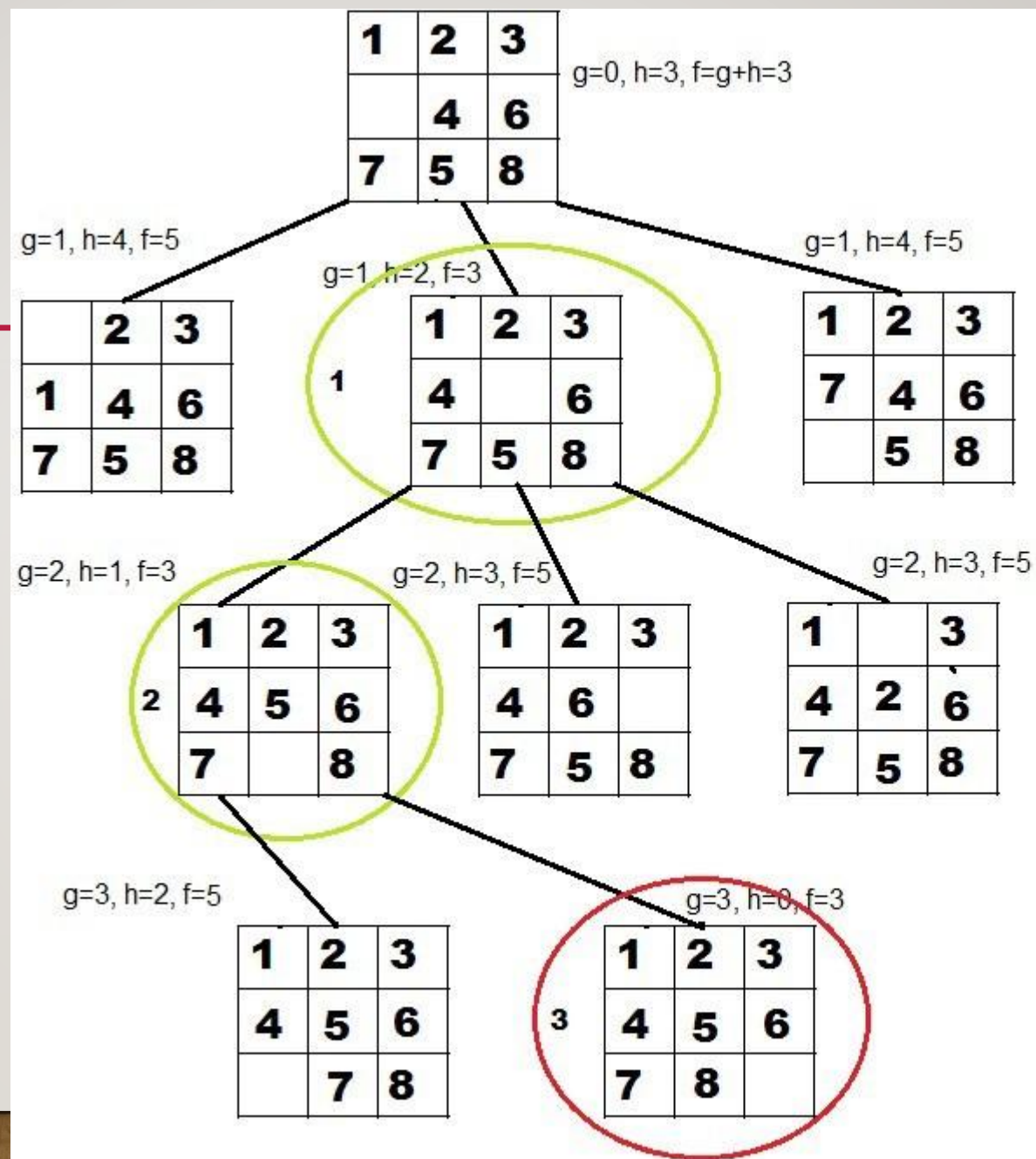
(f) After expanding Pitesti



45

8-PUZZLE

BY A* SEARCH



46 A* SEARCH EXAMPLE

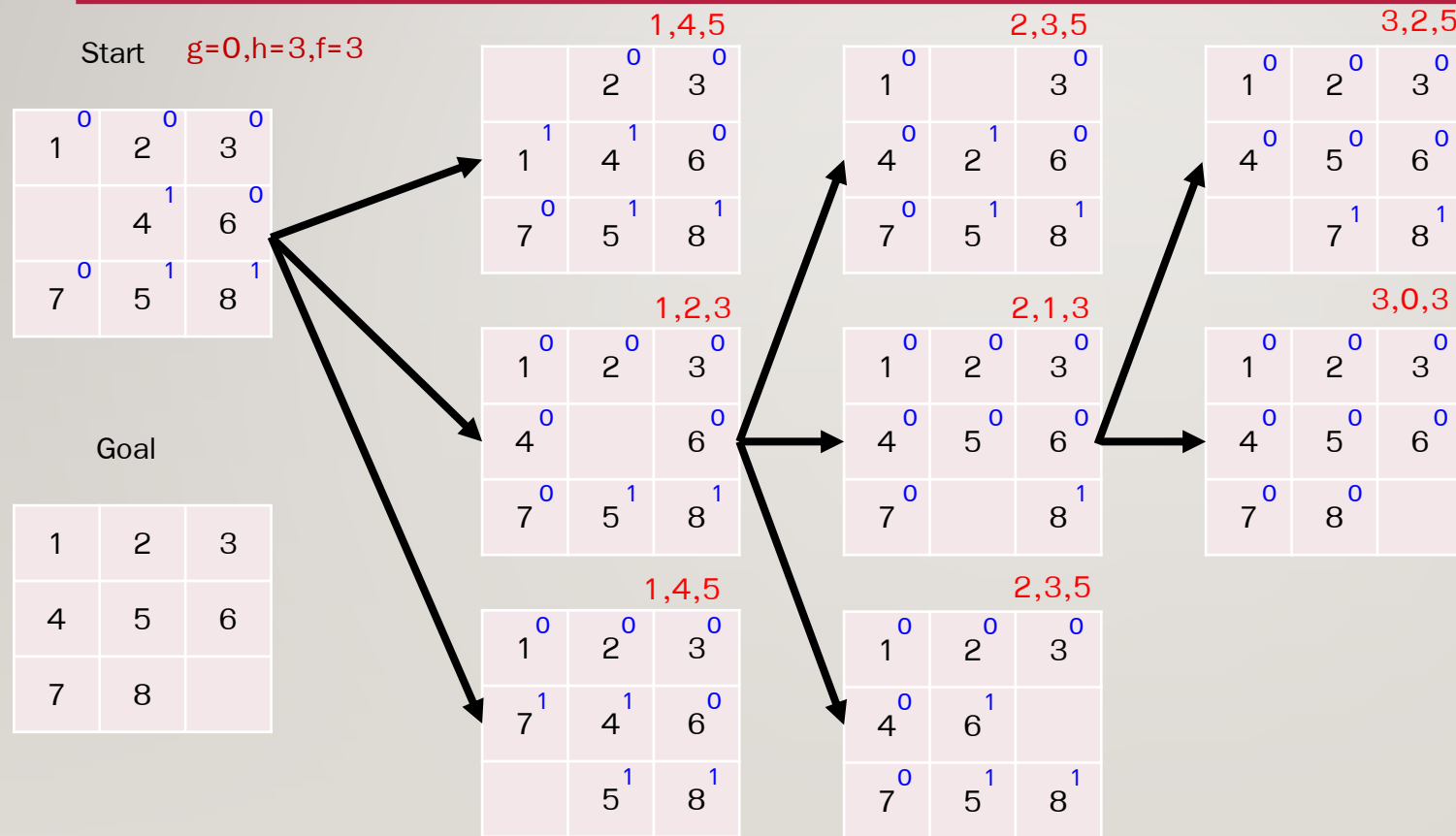
Start

1 ⁰	2 ⁰	3 ⁰
	4 ¹	6 ⁰
7 ⁰	5 ¹	8 ¹

Goal

1	2	3
4	5	6
7	8	

47 A* SEARCH EXAMPLE



48 A* SEARCH EXAMPLE

Can you solve this problem?

